OS -- Basics, Types, Functions & Process Management

[Target] Target: Define OS technically in 20s. Name all 5 process states cold.
Read time: 10 minutes
Mock gap: "Technical definition of Operating System" -- fix this first

WHAT IS AN OPERATING SYSTEM?

Mock feedback gap -- you were asked this and struggled. Memorize this definition.

Technical Definition:

"An operating system is system software that acts as an interface between computer hardware and the user. It manages and controls all hardware resources -- CPU, memory, disk, and I/O devices. It takes instructions from the user, directs the CPU to execute them, and passes results back to the user."

Key roles:
- Interface between user and hardware
- Resource manager (CPU, memory, devices)
- Program execution environment
- Provides services to application software

Your 20-Second Script:

"An OS is software that acts as the interface between user and hardware. It manages CPU, memory, file systems, and I/O devices. Users interact via GUI or CLI -- the OS translates those into hardware instructions. Without an OS, no application can run."

ABSTRACT VIEW -- How OS Sits in the Stack

```
User 1  User 2  ...  User n      Users
Compiler  Assembler  Editor       System Programs
  Database
  Operating System              OS (manages everything below)

  Computer Hardware             CPU, Memory, Disk, I/O
```

TYPES OF OPERATING SYSTEM (8 Types)

Mnemonic (Hinglish): "Batch Mata Real-Data Embedded Network Mobile Multi-processor"

Type
What it does
Example

Batch
Groups similar jobs, runs without user interaction
IBM OS/360

Multi-tasking
Multiple tasks on one CPU, time-sharing
Windows, Linux

Real-time (RTOS)
Strict timing guarantees, deterministic response
Flight control, pacemakers

Distributed
Multiple computers appear as single unified system
Apache Hadoop

Embedded

Built into dedicated devices, minimal UI
Car ECU, microwave

Network
Centralized resource sharing over network
Windows Server

Mobile
Optimized for touch, battery, connectivity
Android, iOS

Multi-processor
Manages multiple CPUs simultaneously
Modern server OS

OS SERVICES
The OS provides the following services to users and programs:
```
      User and Other System Programs

GUI        Batch        Command Line
             User Interfaces
             System Calls

Program     I/O Ops   File Sys   Comm   Resource
  Execution                          Alloc
  Error Detection            Protection &amp;amp; Security
            Operating System

              Hardware
```

10 FUNCTIONS OF AN OPERATING SYSTEM

Mnemonic: "M P F D N S C J S C" = Memory Process File Device Network Security Communication Job-Accounting Secondary-Storage Command

#
Function
What it does

1
Memory Management
Allocates/deallocates RAM to processes

2
Process Management
Creates, schedules, terminates processes

3
File Management
Create, delete, organize files and directories

4
Device Management
Controls I/O devices via device drivers

5

Networking
Manages network connections and protocols

6
Security
Authentication, access control, encryption

7
Communication Management
Inter-process and inter-system communication

8
Job Accounting
Tracks CPU time, memory usage per user/process

9
Secondary Storage Management
Manages disk space, partitions, swap space

10
Command Interpretation
Parses and executes user commands (shell)

PROCESS CONCEPT
A process is a program in execution. More than just code -- includes current activity and resources.
Process in Memory -- 4 Segments
```
High Address
        Stack       grows downward
                    function params, return addr, local vars
        Heap        grows upward, dynamic allocation (malloc/new)

        Data        global variables (initialized + uninitialized)

        Text        program code (read-only)

Low Address
```

Segment
Contains
Notes

Text
Program code
Read-only, shared by all instances

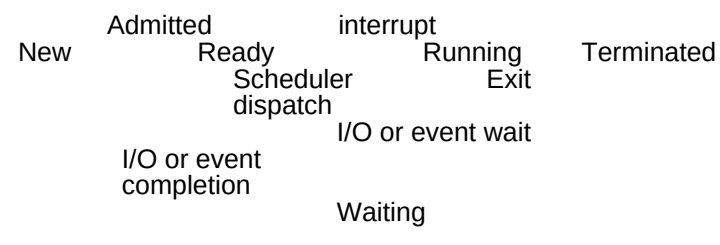
Data
Global variables
Initialized and BSS (uninitialized)

Heap
Dynamic allocation
malloc/new, grows upward

Stack
Function execution
Return addresses, local vars, grows downward

5 PROCESS STATES

Mnemonic (Hinglish): "Naya Redy Running Wait karo Terminat ho gaya"



State
What's happening

New
Process is being created

Ready
Process is in queue, waiting for CPU to be assigned

Running
Instructions are actively executing on CPU

Waiting
Process blocked, waiting for I/O or an event

Terminated
Process has finished execution

State Transitions (Important for interviews):

New → Ready: Admitted (OS accepts process)
Ready → Running: Scheduler dispatch (CPU assigned)
Running → Ready: Interrupt (preempted by OS or higher priority)
Running → Waiting: I/O or event wait (process needs I/O)
Waiting → Ready: I/O or event completion (I/O done, back to queue)
Running → Terminated: Exit (process completed normally)

PROCESS CONTROL BLOCK (PCB)
A PCB (also called Task Control Block) is a data structure in the OS kernel that stores all information about a process.

Every running process has exactly one PCB.

PCB Fields (7 key fields):

Mnemonic: "PP PPC RML" = Pointer, Process-state, Process-number, Program-Counter, Registers, Memory-limits, List-of-files

Field
What it stores

Pointer
Stack pointer -- needed when switching process state

Process State
Current state: new/ready/running/waiting/terminated

Process Number (PID)
Unique process ID assigned by OS

Program Counter
Address of NEXT instruction to execute

Registers
CPU registers (accumulator, base, index, general purpose)

Memory Limits
Page tables, segment tables, memory bounds

List of Open Files
Files currently opened by this process

Your 40-Second Script -- Process & PCB

"A process is a program in execution. In memory it has 4 segments -- Text stores the code, Data stores global variables, Heap is for dynamic allocation, and Stack holds function call data. A process goes through 5 states: New when created, Ready when waiting for CPU, Running when executing, Waiting when blocked on I/O, and Terminated when done.
The OS tracks each process using a PCB -- a data structure containing the PID, program counter, CPU registers, memory limits, and open file list."

Follow-Up Questions (Expect These)
Q: What is the difference between a process and a program?

A program is passive -- just code stored on disk. A process is active -- a program loaded into memory and executing with its own resources (PCB, stack, heap, etc.). The same program can have multiple active processes.

Q: What is context switching?

When the OS switches CPU from one process to another -- it saves the current process's state (into its PCB) and loads the next process's state. The PCB Pointer field is critical here.

Q: What is a zombie process?

A process that has finished execution but its PCB still exists because the parent hasn't read its exit status yet. Also called a "defunct" process.

Q: What is an orphan process?

A process whose parent has terminated but it is still running. The OS re-parents it to the init process (PID 1 on Linux).